VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

ARCHI R KHATRI (1BM24CS051)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by ARCHI R KHATRI **(1BM24CS051)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Syed Akram
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1: Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

int adj[MAX][MAX], visited[MAX], stack[MAX], top = -1;
int n, e;

void dfs(int v) {
   visited[v] = 1;

   for (int i = 0; i < n; i++) {
      if (adj[v][i] && !visited[i]) {
         dfs(i);
      }
   }
   stack[++top] = v;
}
int main() {
   printf("Enter number of vertices and edges: ");
   scanf("%d %d", &n, &e);

   for (int i = 0; i < e; i++) {
      int u, v;
      printf("Enter edge (u v): ");
      scanf("%d %d", &u, &v);
      adj[u][v] = 1;
   }

   for (int i = 0; i < n; i++) {
      visited[i] = 0;
   }

   for (int i = 0; i < n; i++) {
      if (!visited[i]) {
         dfs(i);
      }
   }

   printf("Topological Order: ");
   while (top >= 0) {
      printf("%d ", stack[top--]);
   }

   printf("\n");

   return 0;
}
```

OUTPUT:

```
.\Topologocal }
Enter number of vertices and edges: 6 6
Enter edge (u v): 5 2
Enter edge (u v): 5 0
Enter edge (u v): 4 0
Enter edge (u v): 4 1
Enter edge (u v): 2 3
Enter edge (u v): 3 1
Topological Order: 5 4 2 3 1 0
```

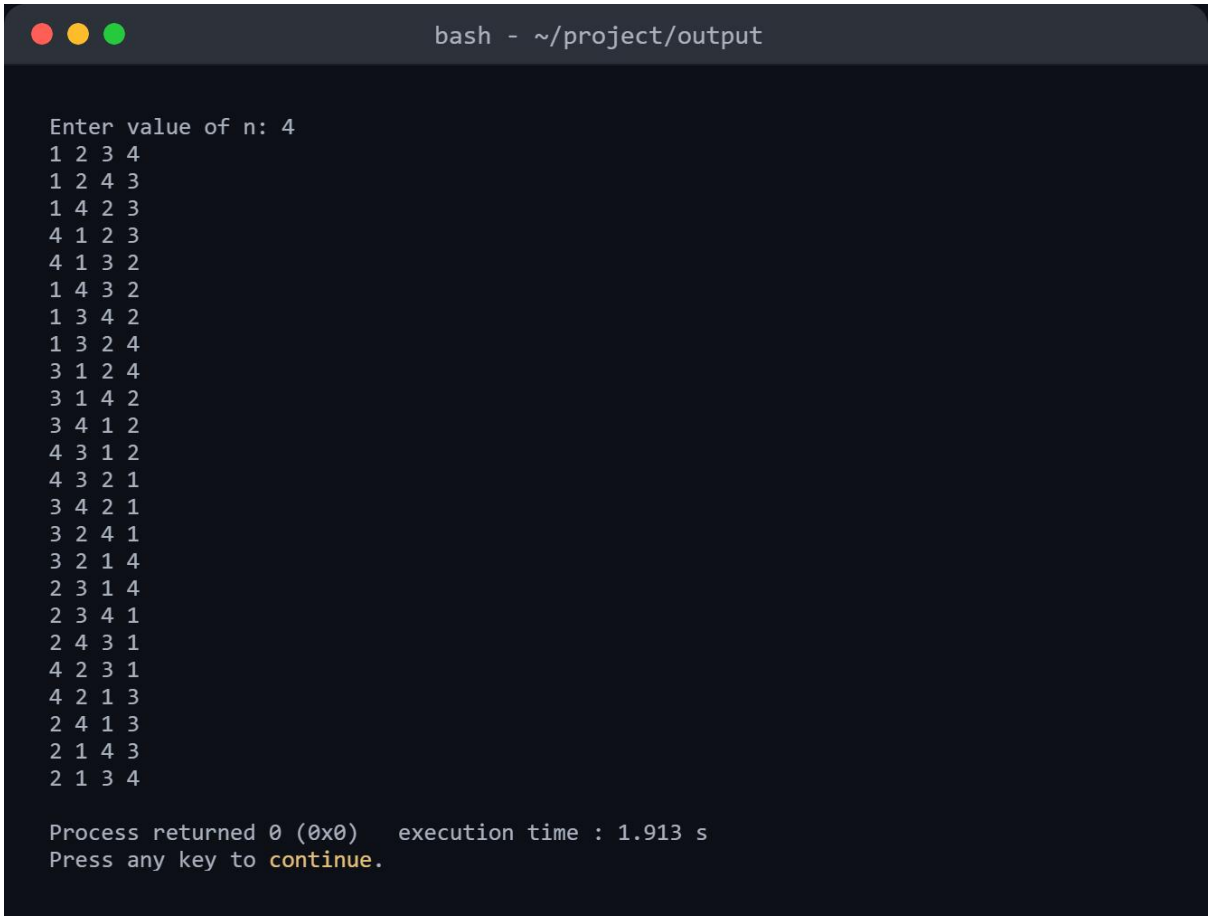
Lab Program 2: Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1

void printPermutation(int perm[], int n) {
   for(int i = 0; i < n; i++) {
      printf("%d ", perm[i]);
   }
   printf("\n");
}
int getMobile(int perm[], int dir[], int n) {
   int mobile = 0;
   int mobileIndex = -1;

   for(int i = 0; i < n; i++) {
      if(dir[perm[i] - 1] == LEFT && i != 0) {
         if(perm[i] > perm[i - 1] && perm[i] > mobile) {
            mobile = perm[i];
            mobileIndex = i;
         }
      }
      if(dir[perm[i] - 1] == RIGHT && i != n - 1) {
         if(perm[i] > perm[i + 1] && perm[i] > mobile) {
            mobile = perm[i];
            mobileIndex = i;
         }
```

```
        }
    }
    return mobileIndex;
}
void johnsonTrotter(int n) {
    int perm[n];
    int dir[n];
    for(int i = 0; i < n; i++) {
        perm[i] = i + 1;
        dir[i] = LEFT;
    }
    printPermutation(perm, n);
    while(1) {
        int mobileIndex = getMobile(perm, dir, n);
        if(mobileIndex == -1)
            break;
        int mobile = perm[mobileIndex];

        if(dir[mobile - 1] == LEFT) {
            int temp = perm[mobileIndex];
            perm[mobileIndex] = perm[mobileIndex - 1];
            perm[mobileIndex - 1] = temp;
            mobileIndex--;
        }
        else {
            int temp = perm[mobileIndex];
            perm[mobileIndex] = perm[mobileIndex + 1];
```

```
            perm[mobileIndex + 1] = temp;
            mobileIndex++;
        }
        for(int i = 0; i < n; i++) {
            if(perm[i] > mobile) {
                dir[perm[i] - 1] *= -1;
            }
        }
        printPermutation(perm, n);
    }
}
int main() {
    int n;
    printf("Enter value of n: ");
    scanf("%d", &n);
    johnsonTrotter(n);
    return 0;
}
```

OUTPUT:


bash - ~/project/output
Enter value of n: 4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4
Process returned 0 (0x0) execution time : 1.913 s
Press any key to continue.

Lab Program 3: Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include<stdio.h>
#include<time.h>
void merge(int arr[],int left,int mid,int right){
   int i,j,k;
   int nl=mid-left+1;
   int nr=right-mid;
   int L[nl],R[nr];
   for(i=0;i<nl;i++) L[i]=arr[left+i];
   for(j=0;j<nr;j++) R[j]=arr[mid+1+j];
   i=0;
   j=0;
   k=left;
   while(i<nl && j<nr){
      if(L[i]<R[j]){
         arr[k++]=L[i++];
      }
      else{
         arr[k++]=R[j++];
      }
   }
   while(i<nl){
      arr[k++]=L[i++];
   }
   while(j<nr){
```

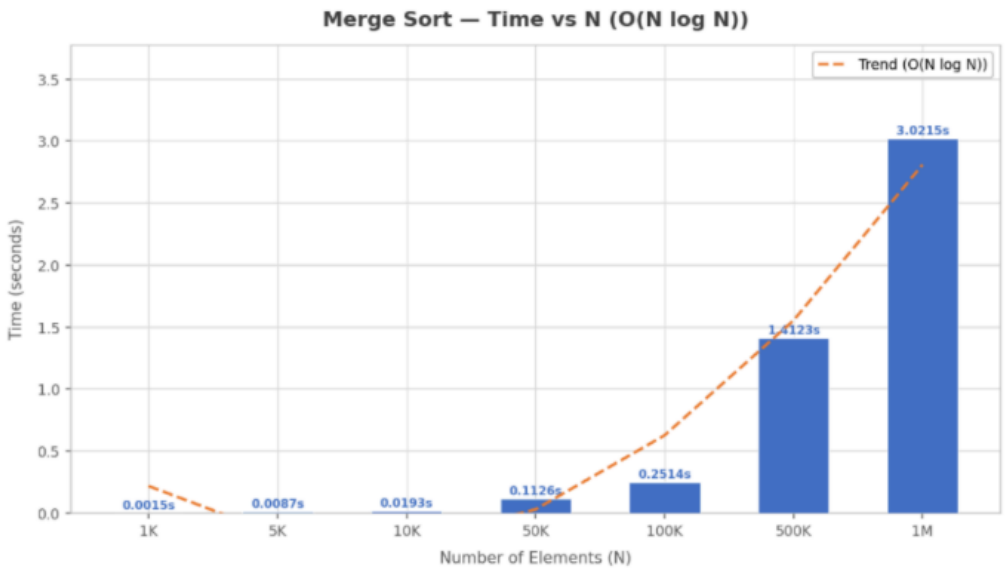
```
        arr[k++]=R[j++];
    }
}
void mergeSort(int arr[],int left,int right){
    if(left<right){
        int mid;
        mid=(left+right)/2;
        mergeSort(arr,left,mid);
        mergeSort(arr,mid+1,right);
        merge(arr,left,mid,right);
    }
}
int main(){
    int n,i;
    clock_t start,end;
    double CPU_time;
    printf("Enter number of elements: ");
    scanf("%d",&n);

    int arr[n];
    printf("Enter the array elements: ");
    for(i=0;i<n;i++)
        scanf("%d",&arr[i]);
    start = clock();
    mergeSort(arr,0,n-1);
    end = clock();
    CPU_time = ((double)(end-start))/CLOCKS_PER_SEC;
```

```
    printf("\nArray after sorting:\n");
    for(i=0;i<n;i++)
        printf("%d\t",arr[i]);
    printf("\nExecution Time: %f seconds\n",CPU_time);
    return 0;
}
```

Output:

```
bash - ~/project/output

Enter number of elements:4
Enter the array elements:9
2
4
3
Array after sorting:2   3       4       9
Process returned 0 (0x0)   execution time : 8.202 s
Press any key to continue.


Enter number of elements:7
Enter the array elements:5 6 1 2 3 0 7
Array after sorting:0   1       2       3       5       6       7
Process returned 0 (0x0)   execution time : 35.221 s
Press any key to continue.
```

Graph:



Lab Program 4: Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

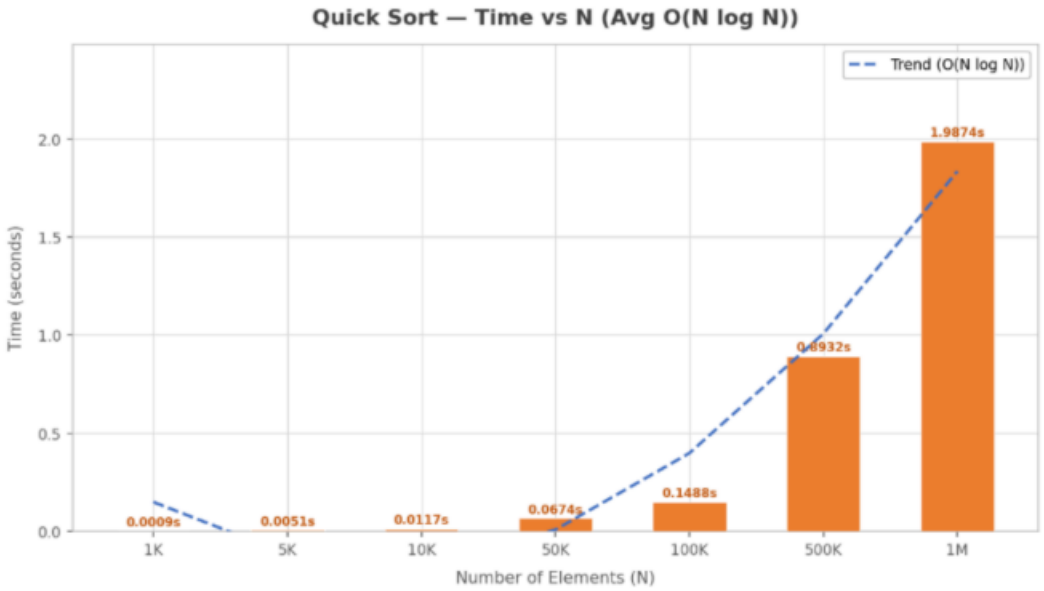
```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
void swap(int* a, int* b){
   int temp=*a;
   *a=*b;
   *b=temp;
}
int partition(int arr[],int low,int high){
   int randomIndex = low + rand() % (high - low + 1);
   swap(&arr[low], &arr[randomIndex]);
   int pivot=arr[low];
   int i=low+1;
   int j=high;

   while(1){
      while(i<=high && arr[i]<=pivot) i++;
      while(arr[j]>pivot) j--;
      if(i>=j) break;
      swap(&arr[i],&arr[j]);
   }
   swap(&arr[j],&arr[low]);
   return j;
}
void quickSort(int arr[],int low,int high){
   if(low<high){
      int p=partition(arr,low,high);
```

```
        quickSort(arr,low,p-1);
        quickSort(arr,p+1,high);
    }
}
int main(){
    int i,n,k;
    printf("Enter Number of elements:");
    scanf("%d",&n);
    int arr[n],temp[n];
    printf("Enter elements:\n");
    for(i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    srand(time(NULL));
    clock_t start=clock();
    for(k=0;k<50000;k++){
        for(i=0;i<n;i++) temp[i]=arr[i];
        quickSort(temp,0,n-1);
    }
    clock_t end=clock();
    double time_taken=((double)(end-start))/CLOCKS_PER_SEC;
    printf("\nSorted array");
    for(i=0;i<n;i++){
        printf("\t%d\t",temp[i]);
    }
    printf("\nTime taken=%f seconds",time_taken);
    return 0;
}
```

OUTPUT:

```
($?) { gcc
Quick_Sort_Modified.c -o Quick_Sort_Modified } ; if ($?) { .\Quick_Sort_Modified }
Enter Number of elements:5
Enter elements:
6
2
0
1
3

Sorted array    0               1               2               3               6
Time taken=0.004000 seconds
```

Graph:



Lab Program 5: Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>
#include <time.h>
void heapcom(int a[], int n)
{
   int i, j, k, item;
   for(i = 1; i <= n; i++)
   {
      item = a[i];
      j = i;
      k = j / 2;
      while(k != 0 && item > a[k])
      {
         a[j] = a[k];
         j = k;
         k = j / 2;
      }
      a[j] = item;
   }
}
void adjust(int a[], int n)
{
   int item, i, j;
   j = 1;
   item = a[j];
   i = 2 * j;
   while(i <= n)
   {
      if((i + 1) <= n)
```

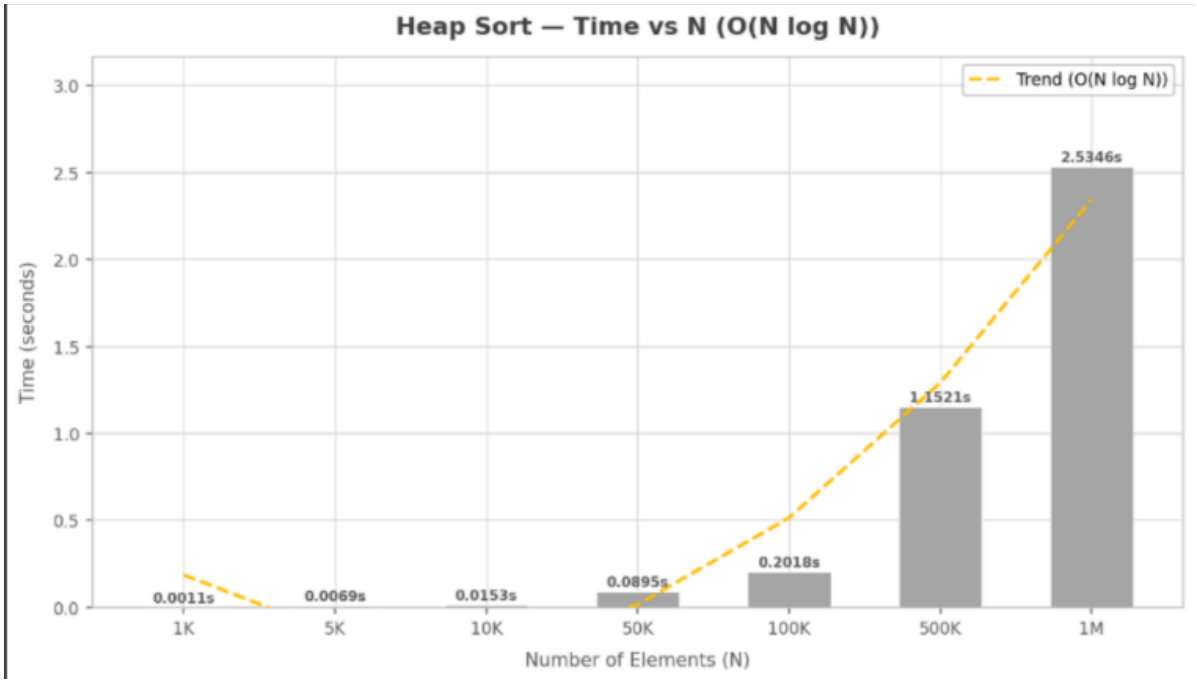
```
        {
            if(a[i] < a[i + 1])
                i++;
        }
        if(item < a[i])
        {
            a[j] = a[i];
            j = i;
            i = 2 * j;
        }
        else
            break;
    }
    a[j] = item;
}
void heapsort(int a[], int n)
{
    int i, temp;
    heapcom(a, n);
    for(i = n; i >= 1; i--)
    {
        temp = a[1];
        a[1] = a[i];
        a[i] = temp;
        adjust(a, i - 1);
    }
}
```

```
int main()
{
   int i, n, a[21], ch = 1;
   clock_t start, end;
   while(ch)
   {
      printf("\nEnter number of elements:\n");
      scanf("%d", &n);
      printf("\nEnter elements:\n");
      for(i = 1; i <= n; i++)
         scanf("%d", &a[i]);
      start = clock();
      heapsort(a, n);
      end = clock();
      printf("\nSorted elements are:\n");
      for(i = 1; i <= n; i++)
         printf("%d\n", a[i]);
      printf("\nTime taken = %lf seconds\n",
            ((double)(end - start)) / CLOCKS_PER_SEC);
      printf("\nRun again? (0/1): ");
      scanf("%d", &ch);
   }
   return 0;
}
```

OUTPUT:

```
C:\Users\bmscecse\Desktop\I

Enter number of elements:
20

Enter elements:
1 2 3 4 5 6 8 0 10 45 55 12 13 10 45 16 78 56 13 20

Sorted elements are:
0 1 2 3 4 5 6 8 10 10 12 13 13 16 20 45 45 55 56 78
Time taken = 0.000000 seconds

Run again? (0/1): 0

Process returned 0 (0x0)   execution time : 29.621 s
Press any key to continue.
```

Graph:

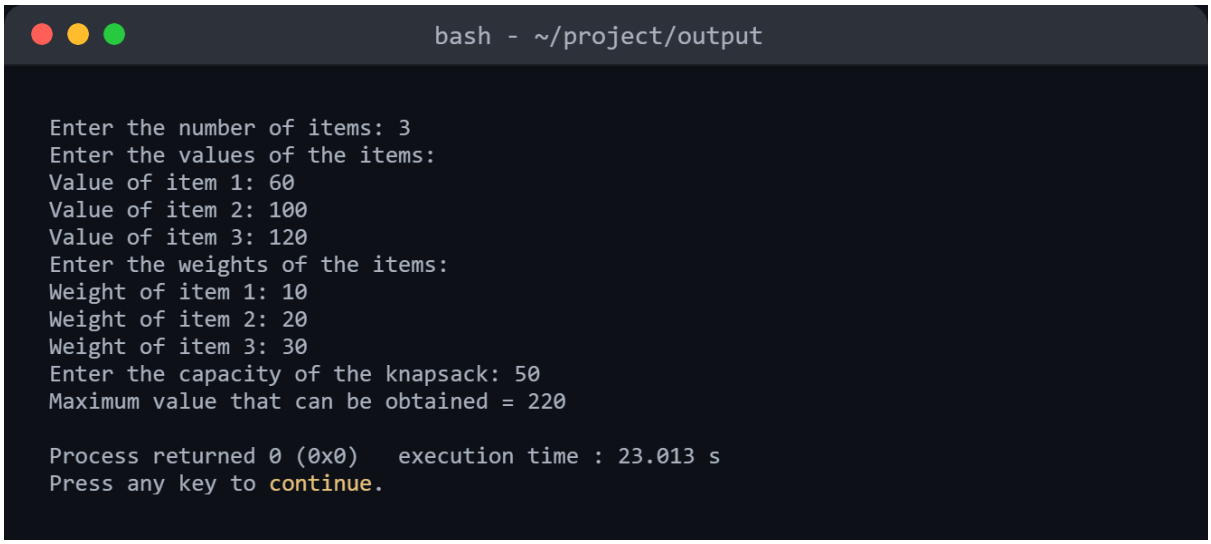


Lab Program 6: Implement 0/1 Knapsack problem using dynamic programming.

```
#include<stdio.h>
int max(int a,int b)
{
   if(a>b)
      return a;
   return b;
}
int main()
{
   int p[100],w[100],M,n,table[100][100];
   printf("Enter the number of items:");
   scanf("%d",&n);
   printf("Enter the maximum capacity of Knapsack:");
   scanf("%d",&M);
   for(int i=1;i<=n;i++){
      printf("Enter the weight and profit of item %d:",i);
      scanf("%d %d",&w[i],&p[i]);
   }
   for(int i=0;i<=n;i++){
      for(int j=0;j<=M;j++)
         if(i==0 || j==0) {
            table[i][j]=0;
         }
         else if(j<w[i]) {
            table[i][j]=table[i-1][j];
         }
         else {
            table[i][j]=max(table[i-1][j],p[i]+table[i-1][j-w[i]]);
```

```
        }
    }
    printf("Maximum profit made:%d",table[n][M]);
    return 0;
}
```

Output:



Lab Program 7: Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include<stdio.h>
#define INF 99

void Floyds(int a[10][10],int n)
{   for(int k=0;k<n;k++)
    {
      for(int i=0;i<n;i++)
      {
        for(int j=0;j<n;j++)
           if(a[i][k]+a[k][j]<a[i][j])
              a[i][j]=a[i][k]+a[k][j];
      }
    }
    printf("Shortest Distance Matrix:\n");
    for(int i=0;i<n;i++)
    {
       for(int j=0;j<n;j++)
           if(a[i][j]==INF)
              printf("INF");
           else
              printf("%d ",a[i][j]);
      printf("\n");
    }
}

int main()
{
    int i,j,k,n;
```

```
    int cost[10][10],a[10][10];
    printf("Enter the number of Vertices:");
    scanf("%d",&n);
    printf("Enter the cost matrix:\n");
    for(i=0;i<n;i++)
    {
       for(j=0;j<n;j++)
       {
          scanf("%d",&cost[i][j]);
          a[i][j]=cost[i][j];
       }
    }
    Floyds(a,n);
    return 0;
}
```

OUTPUT:

```
($?) { gcc Floyd.c -o
Floyd } ; if ($?) { .\Floyd }
Enter number of vertices: 4
Enter the adjacency matrix:
(Use 99999 for INF / no direct edge)
0 99999 3 99999
2 0 99999 99999
99999 7 0 1
6 99999 99999 0

Shortest distance matrix:
       0      10       3       4
       2       0       5       6
       7       7       0       1
       6      16       9       0
```

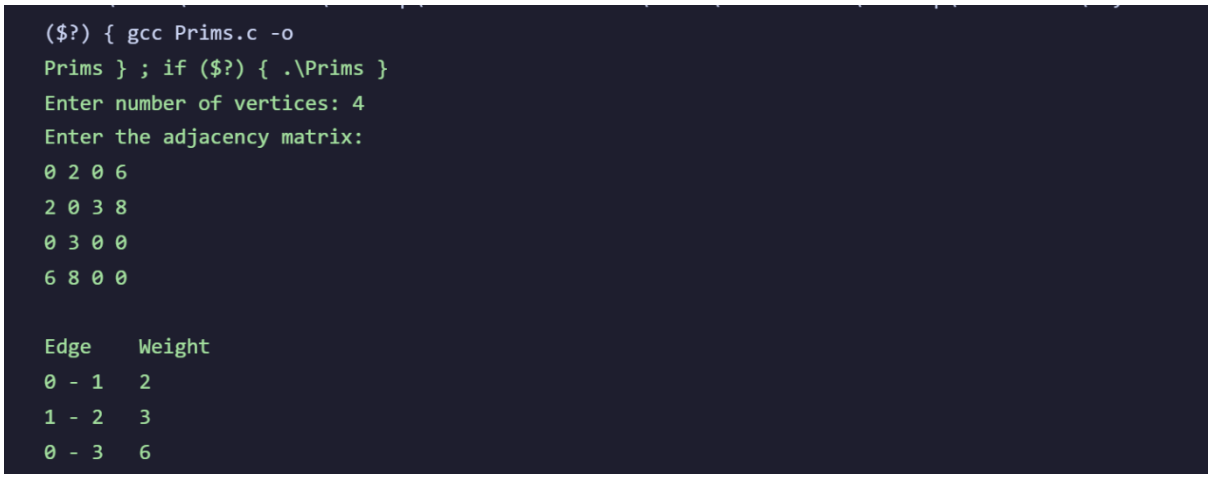
Lab Program 8:(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>
#include <time.h>

#define MAX INT_MAX

int minKey(int key[], int set[], int n)
{
   int min = MAX, min_id;
   for (int i = 0; i < n; i++)
   {
      if (set[i] == 0 && key[i] < min)
      {
         min = key[i];
         min_id = i;
      }
   }
   return min_id;
}
int main()
{
   int n, parent[100], key[100], set[100], graph[100][100];
   int mincost = 0;
   clock_t start, end;
   printf("Enter the number of vertices in the graph: ");
   scanf("%d", &n);
   printf("Enter the adjacency matrix:\n");
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
```

```
    {
        scanf("%d", &graph[i][j]);
    }
}
start = clock();

for (int i = 0; i < n; i++)
{
    key[i] = MAX;
    set[i] = 0;
}
key[0] = 0;
parent[0] = -1;

for (int count = 0; count < n - 1; count++)
{
    int i = minKey(key, set, n);
    set[i] = 1;
    for (int j = 0; j < n; j++)
    {
        if (graph[i][j] &&
            set[j] == 0 &&
            graph[i][j] < key[j])
        {
            parent[j] = i;
            key[j] = graph[i][j];
        }
    }
end = clock();
printf("\nEdge\tWeight\n");
for (int i = 1; i < n; i++)
{
```

```
        printf("%d - %d\t%d\n",parent[i],i, graph[parent[i]][i]);
        mincost += graph[parent[i]][i];
    }
    printf("\nMinimum cost is %d\n", mincost);
    double time = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Execution time: %f\n", time);
    return 0;
}
```

OUTPUT:



Lab Program 8:(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include<stdio.h>

typedef structs
{
  int u,v,w;
}Edge;

Edge edges[100],result[100];
int parent[100];

int find(int i)
{
   while(parent[i]!=i)
   {
      i=parent[i];
   }
   return i;
}
void unionSet(int i,int j)
{
   parent[i]=j;
}
void sortEdge(int e)
{
   int i,j;
   Edge temp;
   for(i=0;i<e-1;i++){
      for(j=0;j<e-1-i;j++)
      {
         if(edges[j].w>edges[j+1].w){
```

```
            temp=edges[j];
            edges[j]=edges[j+1];
            edges[j+1]=temp;
        }
    }
  }
}
int kruskals(int n,int e)
{
  int count=0;
  int cost=0;
  int u,v;
  for(int i=0;i<n;i++){
    parent[i]=i;
  }
  sortEdge(e);
  for(int i=0;i<e&& count<n-1;i++)
  {
    u=find(edges[i].u);
    v=find(edges[i].v);
    if(u!=v){
      result[count++]=edges[i];
      cost+=edges[i].w;
      unionSet(u,v);
    }
  }
  printf("Minimum Spanning Tree:\n");
  for(int i=0;i<count;i++){
    printf("%d---%d = %d\n",result[i].u,result[i].v,result[i].w);
  }
  return cost;
}
```

OUTPUT:

```
($?) { gcc Kruskal.c -o Kruskal } ; if ($?) { .\Kruskal }
Enter number of vertices: 4
Enter number of edges: 5

Enter Source Destination Weight for each edge:
Edge 1: 0 1 10
Edge 2: 0 2 6
Edge 3: 0 3 5
Edge 4: 1 3 15
Edge 5: 2 3 4

Edges in Minimum Spanning Tree:
2 -- 3  Weight = 4
0 -- 3  Weight = 5
0 -- 1  Weight = 10

Minimum Cost = 19
```

Lab Program 9: Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>

int main() {
    int i, j, n, m;
    int p[50], w[50];
    float x[50];
    int t1, t2;

    printf("Enter the number of objects: ");
    scanf("%d", &n);
    printf("Enter the weights of all objects:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &w[i]);
    }
    printf("Enter the profit of all objects:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &p[i]);
    }
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if ((float)p[i] / w[i] < (float)p[j] / w[j]) {

                t1 = p[i];
                p[i] = p[j];
                p[j] = t1;
```

```
            t2 = w[i];
            w[i] = w[j];
            w[j] = t2;
        }
    }
}
printf("Enter the maximum capacity: ");
scanf("%d", &m);
for (i = 0; i < n; i++) {
    x[i] = 0.0;
}
int rem_cap = m;
for (i = 0; i < n; i++) {
    if (w[i] > rem_cap) {
        break;
    }
    x[i] = 1.0;
    rem_cap -= w[i];
}
if (i < n) {
    x[i] = (float)rem_cap / w[i];
}
float sum = 0;
for (i = 0; i < n; i++) {
    sum = sum + (p[i] * x[i]);
}
printf("Total profit : %f\n", sum);
return 0;
}
```

OUTPUT:


bash - ~/project/output
Enter no of objects:7
Enter weight and profit:2 10
3 5
5 15
7 7
1 6
4 18
1 3
Enter the bag capacity:15
Total weight and profit: 15.00 55.33

Lab Program 10: From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm

```
#include <stdio.h>
#include <time.h>

#define n 5

int minDist(int dist[], int visited[]) {
   int min = 99999, min_index = -1;
   for (int i = 0; i < n; i++) {
      if (visited[i] == 0 && dist[i] <= min) {
         min = dist[i];
         min_index = i;
      }
   }
   return min_index;
}

int dijkstra(int weight[n][n], int src) {
   int dist[n];
   int visited[n];
   for (int i = 0; i < n; i++) {
      dist[i] = 99999;
      visited[i] = 0;
   }
   dist[src] = 0;
   for (int count = 0; count < n - 1; count++) {
      int u = minDist(dist, visited);
      visited[u] = 1;
      for (int v = 0; v < n; v++) {
```

```
            if (!visited[v] && weight[u][v] != 0 && dist[u] != 99999) {
                int new_dist = dist[u] + weight[u][v];
                if (new_dist < dist[v]) {
                    dist[v] = new_dist;
                }
            }
        }
    }
    printf("Vertex \t Distance from source\n");
    for (int i = 0; i < n; i++) {
        printf("%d \t\t %d\n", i + 1, dist[i]);
    }
    return 0;
}
int main() {
    int i, j;
    int graph[n][n];
    clock_t start, end;

    printf("Enter the weight matrix (%dx%d):\n", n, n);
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    int source = 0;
    start = clock();
    dijkstra(graph, source);
    end = clock();
```

```
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Time taken: %f seconds\n", time_taken);

    return 0;
}
```

Output:

```
($?) { gcc Dijsktra.c -o Dijsktra } ; if ($?) { .\Dijsktra }
Enter number of vertices: 6
Enter adjacency matrix:
0 2 3 0 0 0
2 0 0 5 0 0
3 0 0 0 0 1
0 5 0 0 4 0
0 0 0 4 0 1
0 0 1 0 1 0
Enter source vertex: 0
Enter destination vertex: 5

Shortest path: 0 2 5
Total Cost: 4
```

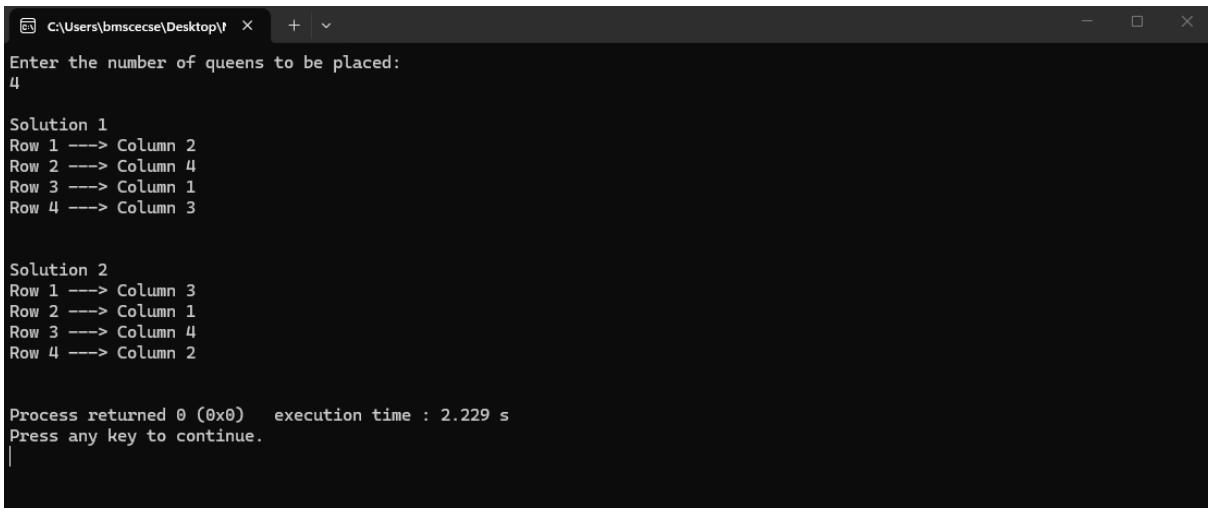
Lab Program 11: Implement "N-Queens Problem" using Backtracking.

```
#include <stdio.h>
#include <stdbool.h>

int solutionCount = 0;

bool isSafe(int N, int board[N][N], int row, int col) {
    int i, j;
    for (i = 0; i < col; i++) {
        if (board[row][i]) {
            return false;
        }
    }
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j]) {
            return false;
        }
    }
    for (i = row, j = col; j >= 0 && i < N; i++, j--) {
        if (board[i][j]) {
            return false;
        }
    }
    return true;
}
void printSolution(int N, int board[N][N]) {
    printf("Solution %d:\n", ++solutionCount);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
```

```
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}
void solveNQueenRecursive(int N, int board[N][N], int col) {
    if (col >= N) {
        printSolution(N, board);
        return;
    }
    for (int i = 0; i < N; i++) {
        if (isSafe(N, board, i, col)) {
            board[i][col] = 1;
            solveNQueenRecursive(N, board, col + 1);
            board[i][col] = 0;
        }
    }
}
void solveNQ(int N) {
    int board[N][N];
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            board[i][j] = 0;
        }
    }
    solveNQueenRecursive(N, board, 0);
    if (solutionCount == 0) {
        printf("Solution does not exist\n");
```

```
    } else {
        printf("Total solutions = %d\n", solutionCount);
    }
}
int main() {
    int N;
    printf("Enter value of N: ");
    scanf("%d", &N);
    if (N <= 0) {
        printf("Enter a positive number.\n");
        return 0;
    }
    solveNQ(N);
    return 0;
}
```

Output:

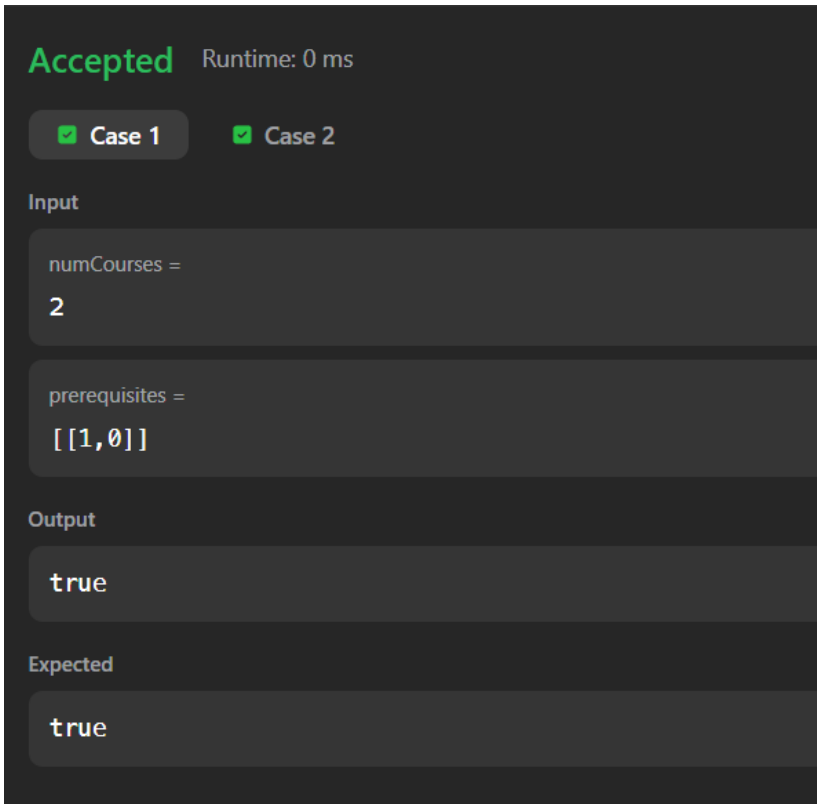


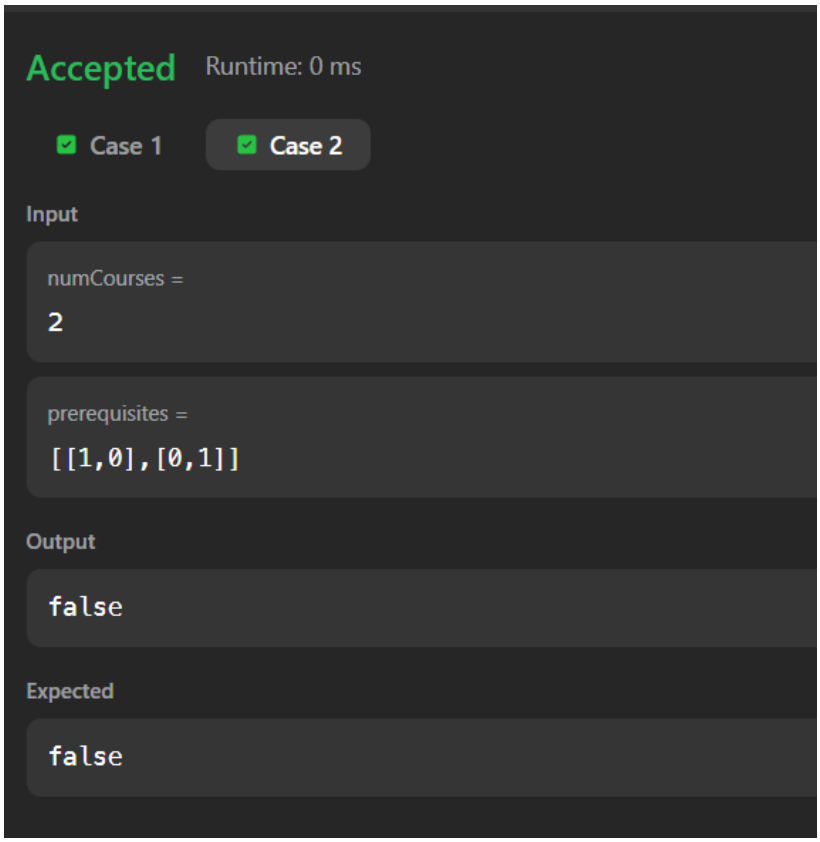
Leetcode: 1)Leet Code exercises on topological sorting(207).

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int inDegree[1000] = {0};
    int queue[1000], front, rear, count;
    front = rear = count = 0;

    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        inDegree[course]++;
    }
    for (int i = 0; i < numCourses; i++) {
        if (inDegree[i] == 0) {
            queue[rear++] = i;
        }
    }
    while (front < rear) {
        int currCourse = queue[front++];
        count++;
        for (int i = 0; i < prerequisitesSize; i++) {
            if (prerequisites[i][1] == currCourse) {
                int depCourse = prerequisites[i][0];
                inDegree[depCourse]--;

                if (inDegree[depCourse] == 0) {
                    queue[rear++] = depCourse;
                }
            }
        }
    }
```

```
    return count == numCourses;
}
```

Output:

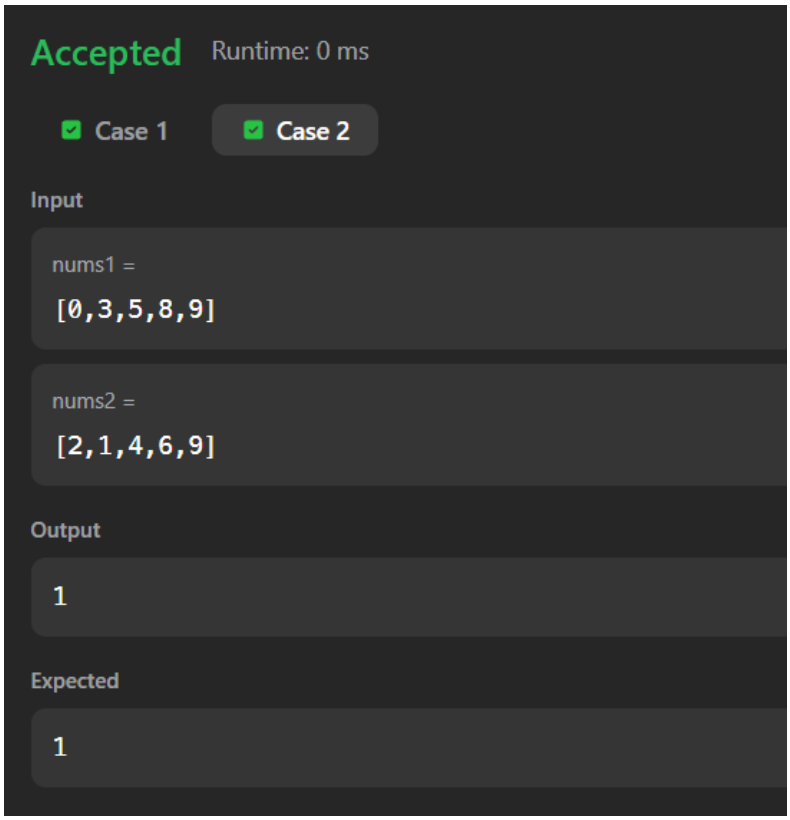


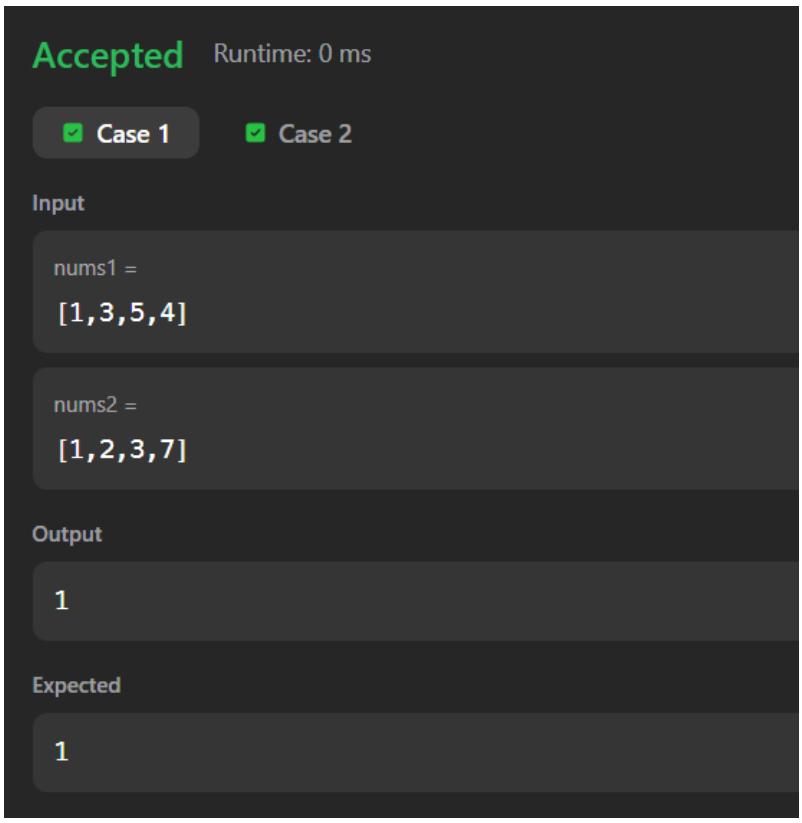


2)Leet Code exercise on Knapsack problem using dynamic Programming(801).

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int x = 0, y = 1;

    for (int i = 1; i < nums1Size; i++) {
        int a = x, b = y;
        if (nums1[i - 1] >= nums1[i] || nums2[i - 1] >= nums2[i]) {
            x = b;
            y = a + 1;
        }
        else {
            y = b + 1;

            if (nums1[i - 1] < nums2[i] &&
                nums2[i - 1] < nums1[i])
                x = (x < b) ? x : b;
                y = (y < a + 1) ? y : (a + 1);
        }
    }
    return (x < y) ? x : y;
}
```

Output:

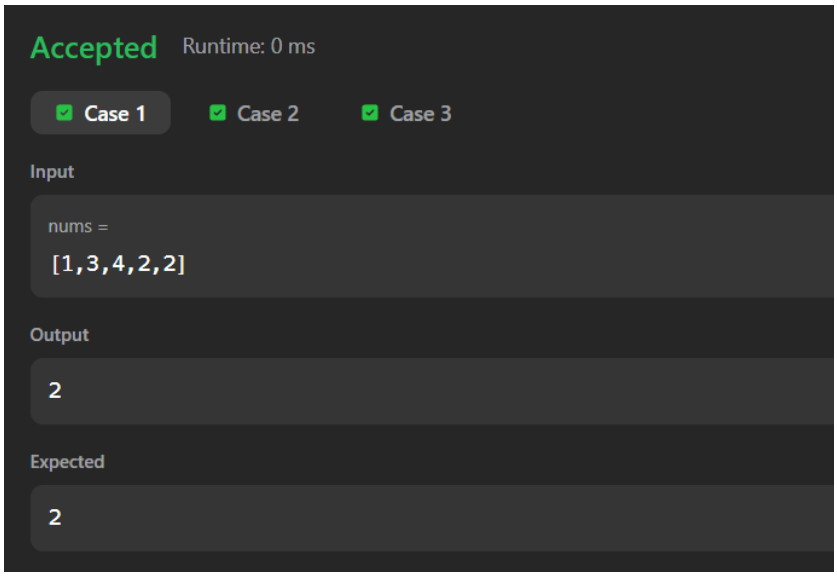

Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums1 =
[0,3,5,8,9]
nums2 =
[2,1,4,6,9]
Output
1
Expected
1

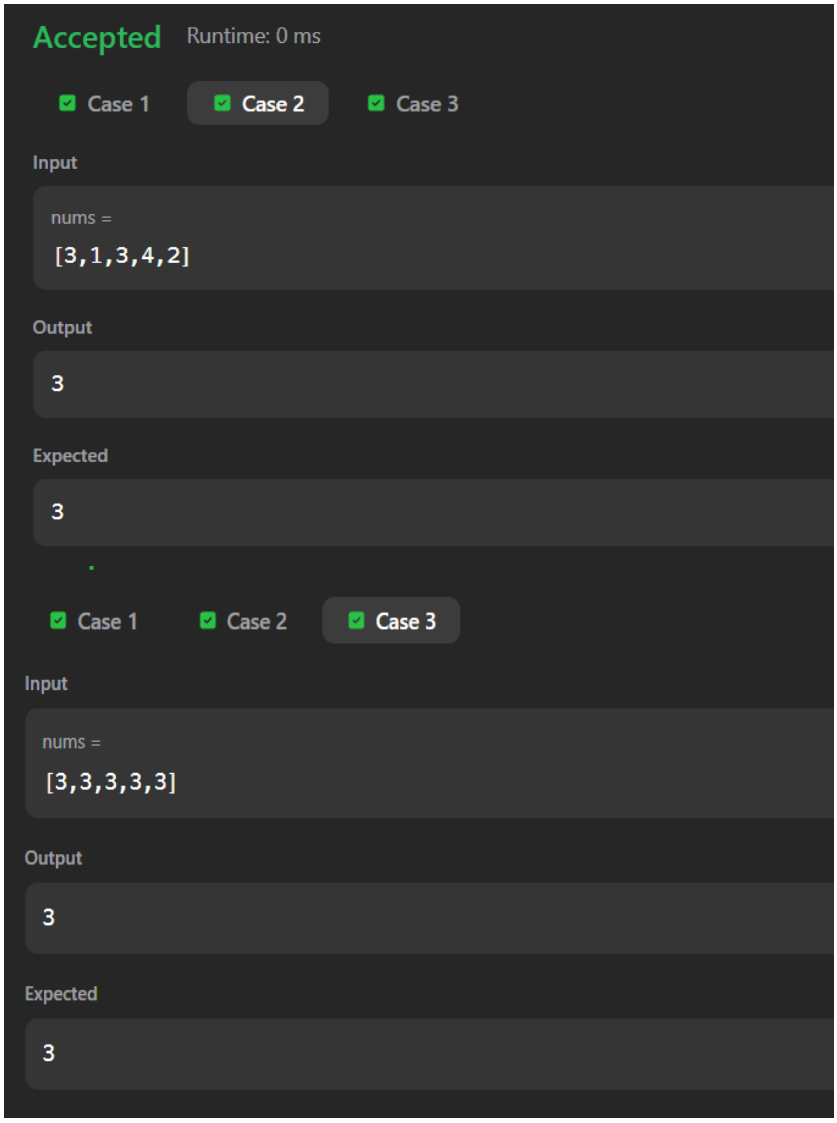

Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums1 =
[1,3,5,4]
nums2 =
[1,2,3,7]
Output
1
Expected
1

3)Leet Code exercise on Floyd's algorithm(287).

```
int findDuplicate(int* nums, int numsSize)
{
   int slow1, fast, slow2;
   slow1=fast=slow2=0;

   while(true)
   {
      slow1=nums[slow1];
      fast=nums[nums[fast]];
      if(slow1==fast)
      {
         break;
      }
   }

   while(true)
   {
      slow1=nums[slow1];
      slow2=nums[slow2];
      if(slow1==slow2)
      {
            return slow2;
      }
   }
}
```

Output:


Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[1,3,4,2,2]
Output
2
Expected
2


Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[3,1,3,4,2]
Output
3
Expected
3
Case 1
Case 2
Case 3
Input
nums =
[3,3,3,3,3]
Output
3
Expected
3